NESNEYE DAYALI PROGRAMLAMA DERS 3

DR. ÖĞRT. ÜYESİ MEHMET BAKIR

Konular

- Diziler
  - Tanımlama
  - Kodlama
  - İlk değer atama
  - Dizi Kullanımı

  - foreach döngüsü
  - Array Sınıfı

  - Çok Boyutlu Diziler
  - Düzensiz Diziler

Aynı Tipten Çok Değişken

- Bazen aynı amaç için birden fazla aynı tip değişkene ihtiyacımız olur.

- Örneğin; Her öğrencinin notunu almak istiyoruz.
  - her biri 0 – 100 arasında birer tam sayı
  - değişkenler: `not1, not2, not3, ...`

  - Bu çok fazla değişken gerektirir.
  - Programın her birini bir bir ele alması gerekecek.

- Daha iyi bir yol var: Dizileri kullanmak

Diziler

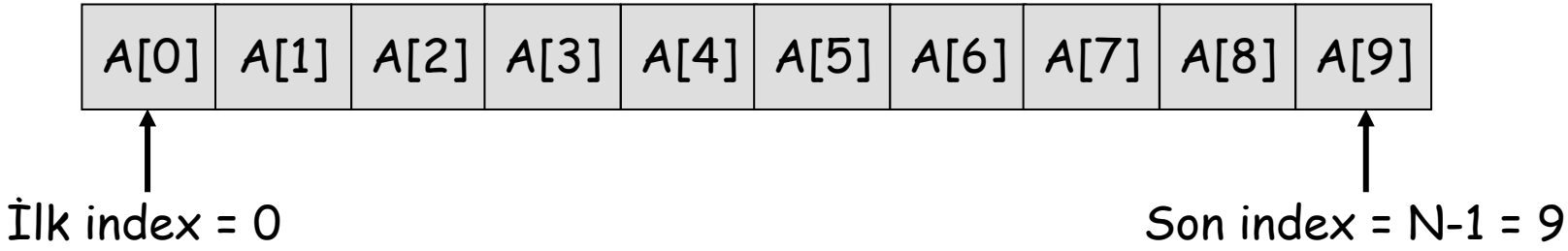
- Birden fazla aynı tip değişkeni bir arada tutan veri yapısıdır.

- En basit tipi bir boyutlu olanıdır.
  - Bir boyutlu dizinin elemanları bir satırda bir biri ardına dizilmiş şekilde kabul edilir.

12	23	34	45	32	17	43	87	96	…

Dizi kodlama

```
int[] A = new int[10];
```



- 10 adet int değişken A ismi ile bir arada tanımlanmıştır.
- Her hangi birine erişmek için index kullanılır
- İlk değişkene erişmek için A[0] kullanılır.
- Son değişkene erişmek için A[9] kullanılır.

Dizilerde varsayılan değerler

```
int[] dizi = new int[25];
```

- Yukarıdaki kod ile int türünden 25 elemanlı dizi adında bir dizi tanımlandı ve dizinin her bir elemanına int türünün varsayılan değer atandı.
- Varsayılan değerler:
  - sayısal türler için 0,
  - object türü için null (yokluk),
  - string türü için null,
  - char için ' ' (boşluk),
  - bool için false değerleridir.

Dizilere İlk Değer Atama

- Dizilere tanımlama sırasında ilk değer atanabilir.
  - Bu durumda dizinin boyutu eleman sayısı kadar olacaktır.

```
int[] A = { 8, 4, 10, 2, 5, 6, 7, 8, 9, 4 };
```

Veya

```
int[] A = new int[] { 8, 4, 10, 2, 5, 6, 7, 8, 9, 4 };
```

- Birden fazla dizi beraber tanımlanabilir.

```
int[] dizi1 = new int[10], dizi2 = new int[20];
int[] dizi3, dizi4;
```

Dizi Kullanımı

- Dizilerin her bir elemanına ulaşmak için her elemanın indexini kullanmamız gerekiyor.

- Bir index elemanın dizideki yerini ifade ediyor.

- Dizinin elemanları peş peşe sıralanmıştır. (arada boşluk yok)

- Dizinin her elamanı sırasıyla tanımlanır ve bu sıralama **0** dan başlar.

Dizi Kullanımı

- Örnek:

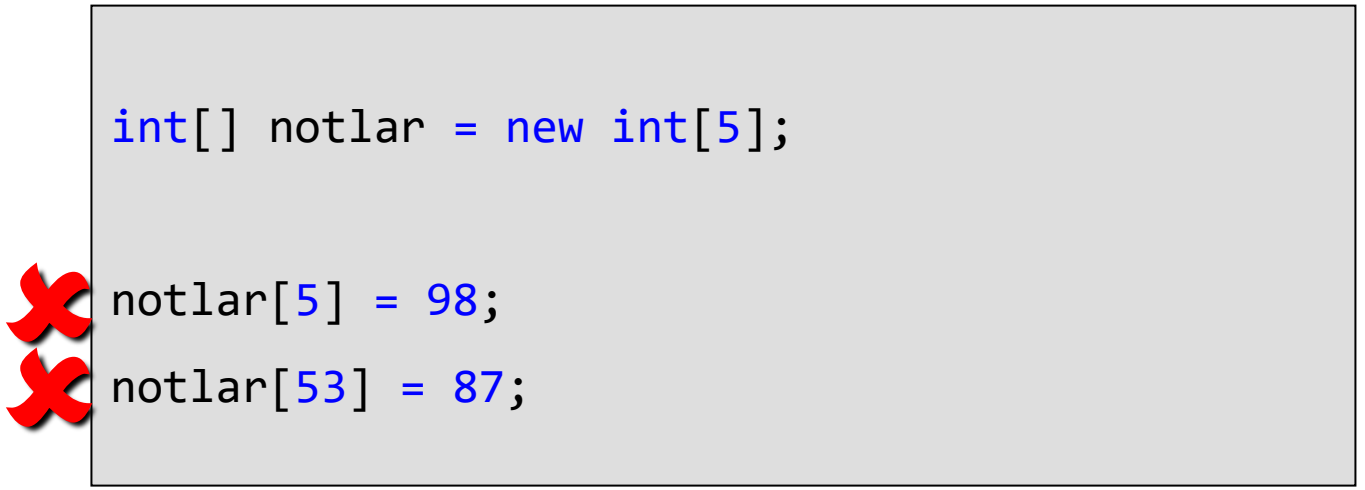
```
int[] notlar = new int[5];



notlar[0] = 98;
notlar[1] = 87;
notlar[2] = 92;
notlar[3] = 79;
notlar[4] = 85;
```

Dizi Kullanımı

- Uyarı!
  - İndexleri kullanırken, indexin doğru aralıkta olup olmadığına dikkat edin!
    - yani, index değerleri, [ ] operatorü kullanılırken dizinin sınırları içerisinde mi değil mi diye bakın!



Dizi Kullanımı

- Dizinin elemanlarına ulaşılırken genelde dönüler kullanılır, ve döngünün her iterasyonunda dizinin bir elemanı üzerinde çalışılır.

- Diziler için en sık kullanılan döngü **`for`** döngüsüdür:
  - Çünkü for döngüsü ifadesinde index'in başlama değeri, ilerlemesi ve sınırı kolaylıkla belirtilebiliyor.

```
notlar[0] = 0;
notlar[1] = 0;
notlar[2] = 0;
notlar[3] = 0;
notlar[4] = 0;
```



```
for(int i = 0; i < 5; i++)
        notlar[i] = 0;
```

Dizi-Örnek, 1/3: Okuma

```
/* klavyeden girilen değerleri diziye atma*/
using System;
class Program
{
   static void Main()
   {
      double[] a = new double[5];
      Console.WriteLine("5 tane dizi elemanı giriniz:");
      /* dizi elemanlarını oku*/
      for (int i = 0; i < 5; i++)
      {
         double.TryParse(Console.ReadLine(), out a[i]);
      }
   }
}
```

```
5 tane dizi elemanı giriniz: 1.2 3.4 5.6 7.8 9.0
```

Dizi-Örnek, 2/3: Yazma

```
// dizinin elemanlarını ekrana yazma
using System;
class Program
{
   static void Main()
   {
      double[] a = { 1.2, 3.4, 5.6, 7.8, 9.0 };

      /* dizinin elemanlarını göster*/
      for (int i = 0; i < 5; i++)
      {
         Console.WriteLine("a[{0}]={1:F2}", i, a[i]);
      }

      Console.ReadLine();
   }
}
```

```
a[0] = 1.20
a[1] = 3.40
a[2] = 5.60
a[3] = 7.80
a[4] = 9.00
```

Dizi-Örnek, 3/3: Max

```
// Dizideki en büyük elemanı bulma
using System;
class Program
{
   static void Main()
   {
      double[] a = new double[5] { 1.2, 3.4,
                                   5.6, 7.8,
                                   9.0 };

      double max = 0.0;
      /* Dizideki en büyük elemanı bul*/
      for (int i = 0; i < 5; i++)
      {
         if (a[i] > max)
            max = a[i];
      }
      Console.WriteLine("max={0:F2}", max);
      Console.ReadLine();
   }
}
```

```
max = 9.00
```

Dizi-Örnek, Bir arada

```
/* klavyeden girilen değerleri diziye atma,
 * dizinin elemanlarını ekrana yazma ve en büyük dizi elemanını bulma*/
using System;
class Program
{
   static void Main()
   {
      double[] a = new double[5];
      Console.WriteLine("5 tane dizi elemanı giriniz:");

      for (int i = 0; i < 5; i++)   /* Dizi elemanlarını oku*/
         double.TryParse(Console.ReadLine(), out a[i]);


      for (int i = 0; i < 5; i++)   /* Dizinin elemanlarını göster*/
         Console.WriteLine("a[{0}]={1:F2}", i, a[i]);

      double max = 0.0;             /* Dizideki en büyük elemanı bul*/
      for (int i = 0; i < 5; i++)
      {
         if (a[i] > max)  max = a[i];
      }
      Console.WriteLine("max={0:F2}", max);
   }
}
```

foreach Döngüsü

- foreach yalnızca dizilere ve listelere (koleksiyonlara) uygulanabilen bir döngü yapısıdır.
  - Kullanımı şu şekildedir:

```
int[] dizi = { 3, 2, 6, 7 };

foreach (int eleman in dizi)
{
   Console.WriteLine(eleman);
}
```

  - Burada dizinin bütün elemanları teker teker ekrana yazdırılıyor.

foreach

- foreach döngüsüyle dizi elemanlarının değerini değiştiremeyiz, sadece ekrana yazdırmak gibi "read-only" işlemler yapabiliriz.

```
int[] dizi = { 3, 2, 6, 7 };

foreach (int eleman in dizi)
{
    eleman*=2;
    Console.WriteLine(eleman);
}
```

Length

- Dizinin boyutunu verir
- Değeri int tipindendir.

```
int[] dizi1 = { 1, 2, 3, 4, 9, 8, 7, 6, 5 };

for (int i = 0; i < dizi1.Length; i++)
   Console.WriteLine(dizi1[i]);
```

- Dizinin her bir elemanını ekrana yazdırır.

CopyTo()

```
int[] dizi1 = { 1, 2, 3, 4 };
int[] dizi2 = new int[10];


dizi1.CopyTo(dizi2, 0);
```

- dizi1 4 elemanlı ve ilk değerleri atanmış,
- dizi2 ise 10 elemanlı ama ilk değerleri 0,
  - Burada 4 tane eleman dizi1'den dizi2'ye kopyalanır. Kopyalama işlemi 0. indeksten başlar.

Max(), Min()

```
int[] dizi1 = { 1, 2, 3, 4, 9, 8, 7, 6, 5 };

int a = dizi1.Max();
int b = dizi1.Min();

Console.WriteLine("Max:" + a + " Min:" + b);
```

- dizi1 in en büyük ve en küçük elemanlarını a ve b değişkenlerine atayacak, daha sonra a ve b değişkenlerini ekrana yazdıracak.

- Dizilerde min ve max metodlarını kullanmak için programın başına:
  - `using Sytem.Linq;`

  eklenmelidir.

Konular

- Diziler
  - Tanımlama
  - Kodlama
  - İlk değer atama
  - Dizi Kullanımı

  - foreach döngüsü
  - Array Sınıfı

  - Çok Boyutlu Diziler
  - Düzensiz Diziler

Array Sınıfı

- Dizi oluşturma, değiştirme, sıralama, kopyalama, arama gibi dizi işlemlerini barındıran bir sınıftır.
- En çok kullanılan metodları:
  - Array.CreateInstance
  - Array.Clear
  - Array.Reverse
  - Array.Sort
  - Array.Copy

Array.CreateInstance()

- İstediğimiz tipte, istediğimiz boyutlarda yeni diziler oluşturmamıza yarar.

```
Array dizi = Array.CreateInstance(typeof(int), 5);
```

- Burada int türünden 5 elemanlı dizi adında bir dizi tanımlandı ve dizinin her bir elemanına int türünün varsayılan değeri atandı.

Array.Clear()

- Dizinin belirtilen indexten sonra n tane elemana varsayılan değer yapar.

```
int[] dizi1 = { 12, 45, 23, 36, 44, 25, 89, 65, 11, 10 };
// dizinin önceki hali
Console.Write("Dizinin Önceki hali\t: ");
for (int i = 0; i < dizi1.Length; i++)
   Console.Write(dizi1[i] + " - ");

Console.WriteLine("\n");

// dizinin bazı elemanlarına varsayılan değer ver
// 3. indexten başlayıp 4 tanesini 0'lar
Array.Clear(dizi1, 3, 4);

Console.Write("Dizinin Sonraki hali\t: ");
for (int i = 0; i < dizi1.Length; i++)
   Console.Write(dizi1[i] + " - ");
```

```
Dizinin Önceki hali     : 12 - 45 - 23 - 36 - 44 - 25 - 89 - 65 - 11 - 10 -

Dizinin Sonraki hali    : 12 - 45 - 23 - 0 - 0 - 0 - 0 - 65 - 11 - 10 -
```

Array.Reverse()

- Dizinin eleman sırasını tersine çevirir.

```
int[] dizi1 = { 12, 45, 23, 36, 44, 25, 89, 65, 11, 10 };
// dizinin önceki hali
Console.Write("Dizinin Önceki hali:");
for (int i = 0; i < dizi1.Length; i++)
   Console.Write(dizi1[i] + " - ");

Console.WriteLine();

// diziyi ters çevir
Array.Reverse(dizi1);

// dizinin ters çevrilmiş hali
Console.Write("Dizinin çevrilmiş hali:");
for (int i = 0; i < dizi1.Length; i++)
   Console.Write(dizi1[i] + " - ");
```

```
Dizinin önceki hali     : 12 - 45 - 23 - 36 - 44 - 25 - 89 - 65 - 11 - 10
Dizinin çevrilmiş hali  : 10 - 11 - 65 - 89 - 25 - 44 - 36 - 23 - 45 - 12
```

Array.Sort()

- Dizinin elemanlarını küçükten büyüğe sıralar

```
int[] dizi1 = { 12, 45, 23, 36, 44, 25, 89, 65, 11, 10 };
// dizinin önceki hali
Console.Write("Dizinin Önceki hali\t:");
for (int i = 0; i < dizi1.Length; i++)
   Console.Write(dizi1[i] + " - ");

Console.WriteLine();

// diziyi küçükten büyüğe sırala
Array.Sort(dizi1);

// dizinin ters çevrilmiş hali
Console.Write("Dizinin sıralanmış hali\t:");
for (int i = 0; i < dizi1.Length; i++)
   Console.Write(dizi1[i] + " - ");
```

```
Dizinin önceki hali     :12 - 45 - 23 - 36 - 44 - 25 - 89 - 65 - 11 - 10
Dizinin sıralanmış hali :10 - 11 - 12 - 23 - 25 - 36 - 44 - 45 - 65 - 89
```

Diziyi Büyükten Küçüğe Sıralama

- Önce diziyi küçükten büyüğe sıralarız
- Sonrada diziyi ters çeviririz.

```
int[] dizi1 = { 12, 45, 23, 36, 44, 25, 89, 65, 11, 10 };
// dizinin önceki hali
Console.Write("Dizinin Önceki hali\t: ");
for (int i = 0; i < dizi1.Length; i++)
   Console.Write(dizi1[i] + " - ");

// diziyi küçükten büyüğe sırala
Array.Sort(dizi1);
// sıralamayı ters çevir
Array.Reverse(dizi1);

// dizinin ters çevrilmiş hali
Console.Write("\nDizinin sonraki hali\t: ");
for (int i = 0; i < dizi1.Length; i++)
   Console.Write(dizi1[i] + " - ");
```

```
Dizinin önceki hali     : 12 - 45 - 23 - 36 - 44 - 25 - 89 - 65 - 11 - 10
Dizinin sonraki hali    : 89 - 65 - 45 - 44 - 36 - 25 - 23 - 12 - 11 - 10
```

Array.Copy()

- Bir diziden başka bir diziye kopyalamayı sağlar.

```
// dizi1 normal olarak oluşturuluyor ve başlangıç değerleri veriliyor.
int[] dizi1 = { 12, 45, 23, 36, 44, 25, 89, 65, 11, 10 };

// dizi2 CreateInstance metodu ile oluşturuluyor
Array dizi2 = Array.CreateInstance(typeof(int), 10);

// dizi1, dizi2 nin içine kopyalanıyor.
Array.Copy(dizi1, dizi2, 10);

// dizi2 nin elemanları ekrana yazdırılıyor.
Console.Write("dizi2 elemanları: ");
for (int i = 0; i < dizi2.Length; i++)
    Console.Write(dizi1[i] + " - ");

Console.ReadLine();
```

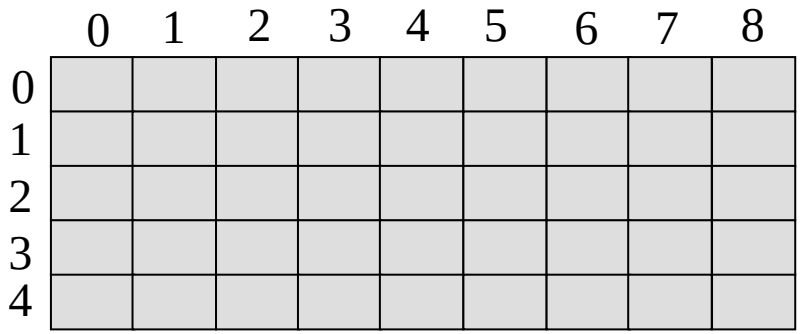
```
dizi2 elemanları: 12 - 45 - 23 - 36 - 44 - 25 - 89 - 65 - 11 - 10 -
```

Çok Boyutlu Diziler

- Bir dizi birden fazla boyutlu olabilir
  - Örneğin, 2-boyutlu dizi (matris) aşağıdaki gibi tanımlanır

```
int[,] M = new int[5, 9];
// 5 satır ve 9 sütundan oluşuyor
```

  - Kavramsal olarak, M dizisi aşağıdakine benzer:



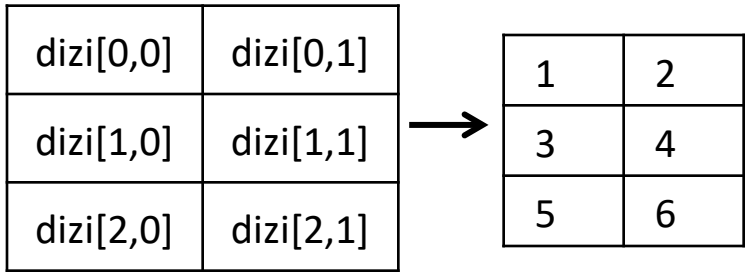
- i satırında ve j sütunundaki elemana ulaşmak için M[i,j] şeklinde yazarız.
  - i satırını, j sütununu seçer.

Çok Boyutlu Dizi Örnek:

```
int[,] dizi = { { 1, 2 }, { 3, 4 }, { 5, 6 } };
```

- Bu dizinin elemanları indekslerine göre aşağıdaki gibidir:
- dizi[0,0] → 1
- dizi[0,1] → 2
- dizi[1,0] → 3
- dizi[1,1] → 4
- dizi[2,0] → 5
- dizi[2,1] → 6

- Bu diziyi matris olarak aşağıdaki gibi gösterebiliriz:



2 Boyutlu Dizilere Ulaşmak

```
/* değer atama */
for (int i = 0; i < 5; i++)
{
   for (int j = 0; j < 9; j++)
   {
      M[i, j] = 0;
   }
}
```

```
/* Toplama */
int toplam = 0;
for (int i = 0; i < 5; i++)
{
   for (int j = 0; j < 9; j++)
   {
      toplam += M[i, j];
   }
}
Console.WriteLine("Toplam =" + toplam);
```

```
/* min ve max bulma */
int min = M[0, 0];
int max = M[0, 0];
for (int i = 0; i < 5; i++)
{
   for (int j = 0; j < 9; j++)
   {
      if (M[i, j] < min) min = M[i, j];
      if (M[i, j] > max) max = M[i, j];
   }
}
Console.WriteLine("min = {0}, max={1}", min, max);
```

Çok Boyutlu Dizilere İlk değer Atama

- Çok boyutlu dizilere ilk değer atamak için iç içe geçmiş bir boyutlu dizi yükleyicilerini kullanabiliriz.

```
int[,] M = { {1, 1, 1, 1, 0, 1, 1, 1, 1},
             {0, 1, 0, 1, 0, 1, 0, 1, 0},
             {1, 0, 0, 1, 1, 1, 0, 0, 1},
             {0, 0, 0, 0, 1, 1, 1, 1, 1},
             {1, 1, 1, 1, 0, 0, 0, 1, 1}};
```

- M dizisi 5 satır ve 9 sütun olacaktır.

Çok Boyutlu Dizi : Örnek

- Diziye verilen ilk değerler ekrana yazdırılıyor.

```
int[,] M = { {1, 1, 1, 1, 0, 1, 1, 1, 1},
             {0, 1, 0, 1, 0, 1, 0, 1, 0},
             {1, 0, 0, 1, 1, 1, 0, 0, 1},
             {0, 0, 0, 0, 1, 1, 1, 1, 1},
             {1, 1, 1, 1, 0, 0, 0, 1, 1} };

for (int i = 0; i < 5; i++)
{
    for (int j = 0; j < 9; j++)
        Console.Write(M[i, j]);
    Console.WriteLine();
}
```

```
111101111
010101010
100111001
000011111
111100011
```

Daha yüksek boyutlu diziler

- Bir dizi birkaç boyutlu olabilir.

```
// ebatları 8 olan bir küp
int[, ,] Kup = new int[8, 8, 8];

// ebatları 4x6x10 olan bir  dikdörtgenler prizması
int[, ,] Prizma = new int[4, 6, 10];

// dört boyutlu bir dizi
float[, , ,] A = new float[5, 5, 5, 5];

Kup[2, 3, 4] = 2;
Prizma[3, 5, 8] = 6;
A[1, 2, 3, 4] = 1234;
```

Düzensiz diziler

- Her bir dizi elemanının farklı sayıda eleman içerebileceği çok boyutlu dizilerdir.

```
int[][] dizi = new int[3][];
dizi[0] = new int[3];
dizi[1] = new int[4];
dizi[2] = new int[2];
```

- Birinci satırda 3 satırı olan ancak sütun sayısı belli olmayan iki boyutlu bir dizi tanımlanıyor.
- İkinci, üçüncü ve dördüncü satırda da bu iki boyutlu dizinin her bir satırının kaç sütun içerdiği ayrı ayrı belirtiliyor.

dizi[0][0]	dizi[0][1]	dizi[0][2]	
dizi[1][0]	dizi[1][1]	dizi[1][2]	dizi[1][3]
dizi[2][0]	dizi[2][1]		

Düzensiz Diziler

- Düzensiz dizilerin elemanlarına, örneğin 0,0 indeksine dizi[0][0] yazarak erişebiliriz.
- Düzensiz dizilerde foreach döngüsü sadece dizi adını yazarak çalışmaz.
  - Ana düzensiz dizinin her bir elemanı için farklı bir foreach döngüsü başlatılmalıdır.
- Şimdiye kadar öğrendiğimiz şekilde düzensiz dizilerin elemanlarını iç içe for döngüsüyle değiştiremeyiz.
  - Çünkü her satır farklı sayıda sütun içerebileceği için satırların sütun sayısı dinamik olarak elde edilmelidir.
  - Bunun için C#'ın System isim alanındaki Array sınıfına ait metotları vardır ve her diziyle kullanılabilirler.

String Sıralama- Sort metodu

```
using System;

class Program
{
    static void Main()
    {
        // isimler dizisi tanımlanıyor
        string[] isimler = new string[] {"Ali", "Veli", "Zeynep", "Cemil",
                                         "Ahmet", "Mehmet", "Oya", "Elif", "Hüsnü" };

        Console.WriteLine("A --> Z Sıralama:");
        // isimler sıralanıyor
        Array.Sort(isimler);
        // sıralanmış dizi ekrana yazdırılıyor.
        foreach (string isim in isimler)
            Console.WriteLine(isim);

        Console.WriteLine("Z --> A Sıralama:");

        // tersinden sıralanıyor
        Array.Reverse(isimler);
        // tersinden sıralanmış dizi ekrana yazdırılıyor.
        foreach (string isim in isimler)
            Console.WriteLine(isim);

        Console.ReadLine();
    }
}
```

```
A --> Z Sıralama:
Ahmet
Ali
Cemil
Elif
Hüsnü
Mehmet
Oya
Veli
Zeynep
Z --> A Sıralama:
Zeynep
Veli
Oya
Mehmet
Hüsnü
Elif
Cemil
Ali
Ahmet
```

Sayı Sıralama – Sort Metodu

```
using System;
class Program
{
    static void Main()
    {
        // isimler dizisi tanımlanıyor
        int[] sayilar = new int[] {12,23,34,45,56,67,32,21,76,56,43,32 };

        Console.WriteLine("Küçükten Büyüğe Sıralama:");
        // isimler sıralanıyor
        Array.Sort(sayilar);
        // sıralanmış dizi ekrana yazdırılıyor.
        foreach (int sayi in sayilar)
            Console.WriteLine(sayi);

        Console.WriteLine("Büyükten Küçüğe Sıralama:");

        // tersinden sıralanıyor
        Array.Reverse(sayilar);
        // tersinden sıralanmış dizi ekrana yazdırılıyor.
        foreach (int sayi in sayilar)
            Console.WriteLine(sayi);

        Console.ReadLine();
    }
}
```

```
Küçükten Büyüğe Sıralama:
12
21
23
32
32
34
43
45
56
56
67
76
Büyükten Küçüğe Sıralama:
76
67
56
56
45
43
34
32
32
23
21
12
```

Konular

- Listeler
  - Tanımlama
  - Eleman ekleme
  - Elemanlara erişme
  - Sık kullanılan metotlar

List<T>

- Türü belirtilerek, indeks ile erişilebilen sınırsız eleman içerebilen bir dizi olarak tanımlanabilir.
  - Boyutları sabit değildir, eleman ekledikçe boyutları dinamik olarak artar.
- Listelerde dizilerdeki gibi sıralama, arama, düzenleme işlemleri yapılabilir.
  - Listelerde daha fazla özgürlük vardır.
- Listeleri kullanabilmek için programın başına aşağıdaki satır eklenmelidir:
  - `using System.Collections.Generic;`

Liste Tanımlama

- List<tip> liste = new List<tip> ();
- List<tip> liste = new List<tip>(kapasite);
- List<tip> liste = new List<tip>(new tip[]{…,…,…});

Şekillerinde tanımlamalar yapılabilir:

```
// liste1'e int türünden değerler eklenebilir
List<int> liste1 = new List<int>();

// liste2'ye float türünden değerler eklenebilir.
// Başlangıç kapasitesi 10 eleman ancak eleman sayısı 10'nu aşınca
// kapasite dinamik olarak arttırılacaktır.
List<float> liste2 = new List<float>(10);

// liste3'e double türünden değerler eklenebilir.
// Başlangıçtaki değerler verilmiş,
// ancak istenilen kadar daha değer eklenebilir.
List<double> liste3 = new List<double>(new double[] { 12, 23, 34, 45, 56, 67, 78 });
```

Listelere eleman ekleme:

- Add()
- AddRange()

Metodları kullanılır:

```
// listeye int türünden değerler eklenebilir
List<int> liste = new List<int>();

// listeye ayrı ayrı üç sayı eklenmektedir.
liste.Add(12);
liste.Add(23);
liste.Add(34);

// listeye bir defada birden çok sayı ekleniyor.
liste.AddRange(new int[] { 45, 56, 67 });

// yeni bir liste tanımlanıyor.
List<int> lst = new List<int>();
lst.Add(1);
lst.Add(2);
lst.Add(3);

// bir listeye aynı türden başka
// bir listenin elemanları eklenebilir.
liste.AddRange(lst);

// bir dizi tanımlanıyor.
int[] dizi = { 12, 23, 45 };

// bir listeye aynı türden
// bir dizinin elemanları eklenebilir.
liste.AddRange(dizi);
```

Listelerin elemanlarına erişme:

- Listelerin elemanlarına index vasıtasıyla erişilebilir:
  - Liste[index]

```
for (int i = 0; i < liste.Count; i++)
{
    Console.WriteLine(liste[i]);
}
```

Count listenin anlık eleman sayısını verir.

```
foreach (int e in liste)
{
    Console.WriteLine(e);
}
```

Liste elamanlarına readonly olarak foreach döngüsü ile de erişilebilir.

ArrayList

- Bu veri tipi boyutu dinamik boyutlandırılır.
- Element eklendikçe kapasitesi büyür.
- Object türünde tutulur.
- Ekleme yaparken dönüşüm işlemi istemez ancak element içindeki değeri çekerken ekleme yapılan tür cinsinden dönüşüme uğrar.
- Eğer veri türünü direkt yönetmek isterseniz ‘List’ yapısını kullanın.

ArrayList - Add

- Add (Ekle): Bu metot ArrayList'in sonuna yeni bir element nesnesi (object) ekler.

```
using System.Collections;

class Program
{
    static void Main()
    {
        //
        // Bir ArrayList oluştur ve üç element ekle.
        //
        ArrayList list = new ArrayList();
        list.Add("One");
        list.Add("Two");
        list.Add("Three");
    }
}
```

ArrayList NameSpace'i

Yeni bir nesne oluşturma
ArrayList bir sınıftır.
New ile bu sınıftan bir nesne oluşturduk.

Örneğin Futbolcu bir sınıf ise
Messi, Ronaldo, Querasma birer nesnedir.
Gelecek haftadan itibaren ayrıntılar başlayacak.

ArrayList - Veri

- ArrayList elementleri bir döngü ile elde edebiliriz ancak her elementin tipini bilmeliyiz. (Dönüşüm işlemi yapacağız.)

```
using System;
using System.Collections;

class Program
{
    static void Main()
    {
        // ArrayList yaratılıyor ve iki int değer ekleniyor.
        ArrayList list = new ArrayList();
        list.Add(5);
        list.Add(7);
        // foreach döngüsü ile
        foreach (int i in list)
        {
            Console.WriteLine(i);
        }
    }
}
```

```
using System;
using System.Collections;

class Program
{
    static void Main()
    {
        // ArrayList yaratılıyor ve iki int değer ekleniyor.
        ArrayList list = new ArrayList();
        list.Add(5);
        list.Add(7);
        // for döngüsü ile
        for (int i = 0; i < list.Count; i++)
        {
            Console.WriteLine(list[i]);
        }

        Console.ReadKey();
    }
}
```

ArrayList

- ArrayList içindeki sayıları toplama
- foreach
- for

```
using System;
using System.Collections;

class Program
{
    static void Main()
    {
        // ArrayList yaratılıyor ve iki int değer ekleniyor.
        ArrayList list = new ArrayList();
        list.Add(5);
        list.Add(7);

        int toplam = 0;
        for (int i = 0; i < list.Count; i++)
        {
            toplam += (int)list[i];
        }
        Console.WriteLine(toplam);
        Console.ReadKey();
    }
}
```

Bilinçli Dönüşüm

Dizi Boyutu verir

```
using System;
using System.Collections;

class Program
{
    static void Main()
    {
        // ArrayList yaratılıyor ve iki int değer ekleniyor.
        ArrayList list = new ArrayList();
        list.Add(5);
        list.Add(7);

        int toplam = 0;
        foreach (int i in list)
        {
            toplam += i;
        }
        Console.WriteLine(toplam);
        Console.ReadKey();
    }
}
```

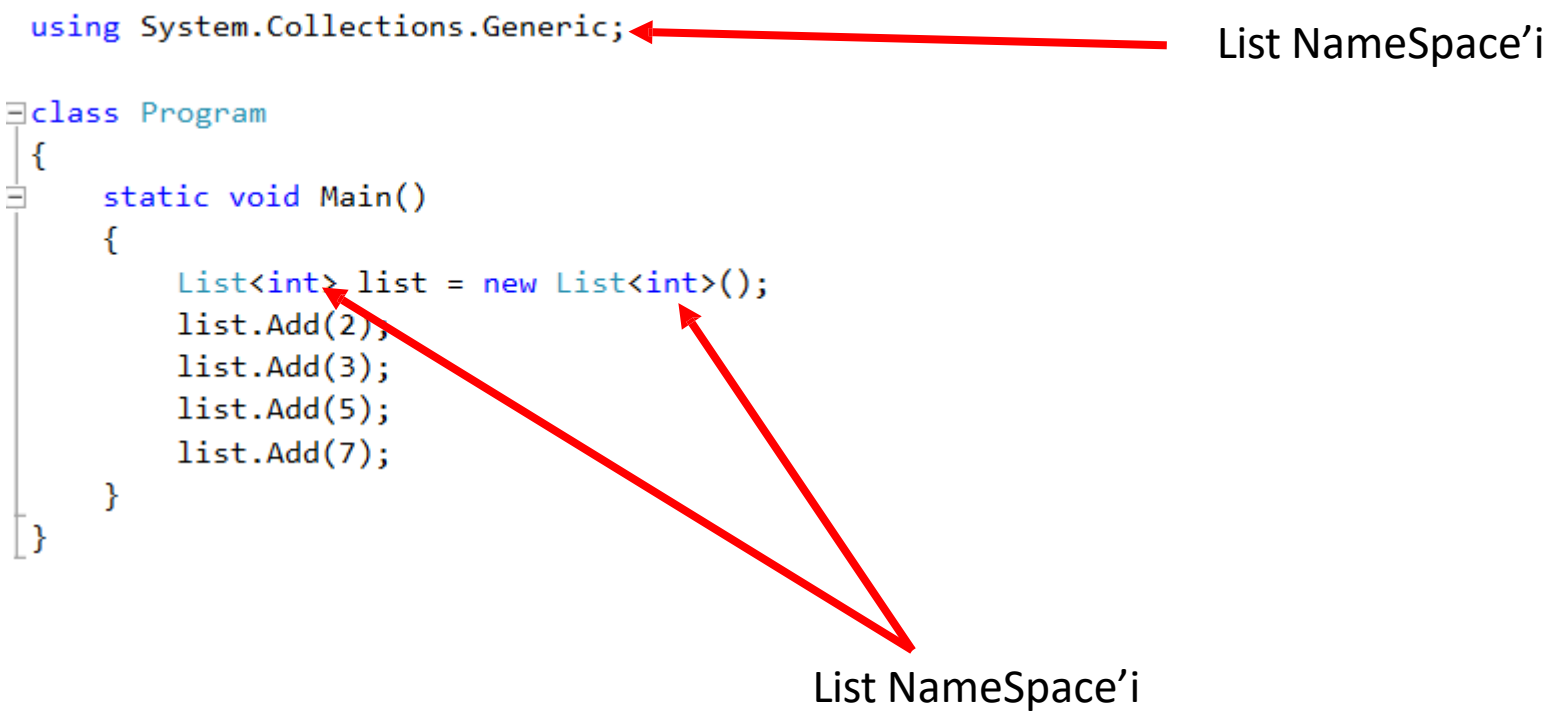
foreach içinde dönüşüm yapılabilir

ArrayList - Diğer Metotlar

- Count: Count ArrayList'in boyutunu verir. Ancak, elemanları teker teker sayarak yapmaz, bunun yerine her işlemden sonra değeri güncellenir. Yani bir eklemede değeri bir artar, bir çıkarmada değeri bir azalır. Bu açıdan hızlı sonuç verir.
- Clear: ArrayList'in tüm elementlerini siler.
- Sort: Sıralama işlemini yapar.
- Reverse: Terse çevirme işlemi yapar.
- GetRange: ArrayList'in istenilen aralıkta veriyi çekebilir ve yeni bir ArrayList döndürür. Sadece ArrayList'in bir kısmı ile ilgilendiğimizde ideal bir metottur.
- SetRange: Mevcut ArrayList'in bir kısmını seçmede kullanılır. GetRange sonuçları yeni bir ArrayList taşırken, SetRange taşımaz.
- IndexOf: Aranan bir element için IndexOf ilk bulduğu değerin pozisyonunu döndürür. Eğer elementi bulamazsa -1 döndürür.
- LastIndexOf: Aranan bir element için son sonucu bulur.
- Convert: ArrayList'inizi Array veri türüne dönüşüm yapabilirsiniz. (Performans artışı sağlar: Sebebi Veri Yapıları dersinde…)

List

- ArrayList'e benzer bir yapıdır ancak veri türü «List» içinde tanımlanır. (Performans artışı sağlar: Sebebi Veri Yapıları dersinde…)



List

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<int> list = new List<int>();
        list.Add(2);
        list.Add(3);
        list.Add(7);

        //iki döngüde aynı işlemi yapar

        foreach (int prime in list) // foreach.
        {
            Console.WriteLine(prime);
        }

        for (int i = 0; i < list.Count; i++) // for.
        {
            Console.WriteLine(list[i]);
        }
    }
}
```

List - Count ve Clear

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<bool> list = new List<bool>();
        list.Add(true);
        list.Add(false);
        list.Add(true);
        Console.WriteLine(list.Count); // 3

        list.Clear();
        Console.WriteLine(list.Count); // 0
    }
}
```

List - Copy

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        int[] arr = new int[3]; // Array
        arr[0] = 2;
        arr[1] = 3;
        arr[2] = 5;
        List<int> list = new List<int>(arr); // List'e kopyala
        Console.WriteLine(list.Count);       // List
    }
}
```

List - IndexOf

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<int> primes = new List<int>(new int[] { 19, 23, 29 });

        int index = primes.IndexOf(23); // Exists.
        Console.WriteLine(index);

        index = primes.IndexOf(10); // Does not exist.
        Console.WriteLine(index);
    }
}
```

List - Contains

List üzerinde arama yapar ve bool sonuç döndürür.

Exist ve Find metotları da arama için kullanılır.

```
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
    static void Main()
    {
        //List oluşturma
        var list = new List<string>();
        list.Add("cat");
        list.Add("dog");
        list.Add("moth");

        //Arama
        if (list.Contains("dog"))
        {
            Console.WriteLine("dog was found");
        }

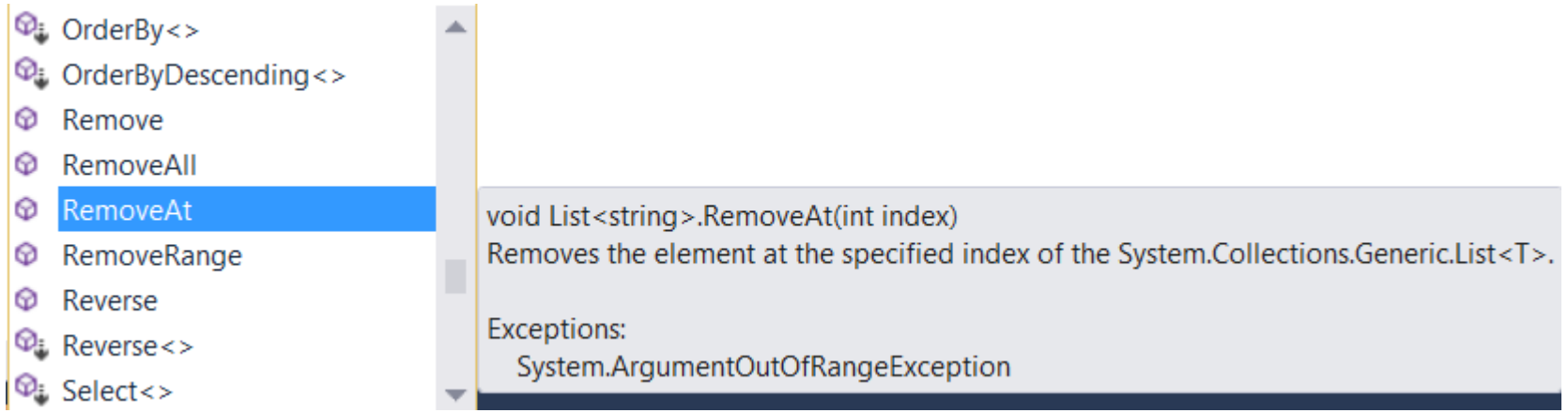
        //Arama case sensitive
        if (list.Contains("MOTH", StringComparer.OrdinalIgnoreCase))
        {
            Console.WriteLine("MOTH was found (insensitive)");
        }

        //Arama sonucu bulunamadı
        Console.WriteLine(list.Contains("fish"));
    }
}
```

List - Diğer Metotlar

- Insert: List'te araya element ekleme
- Remove, RemoveAt, RemoveAll: List'ten element çıkarma

- ArrayList ve List benzer komutlar içerir. Birçok metota sahiptirler. Bu derste bu metotların sadece bir kaçını gördük. Nesnenizi yarattıktan sonra . (noktaya) bastığınızda yardım gelecektir. Örneğin:
  - RemoveAt: bir int index alır. Listedeki pozisyon bilgisi olsa gerek. Void bir metotmuş. Demek ki değer döndürmüyor. Listeden sadece çıkarma işlemini yapıyor.



Listelerde en çok kullanılan metotlar:

- Clear( )
- Contains( )
- CopyTo( )
- IndexOf( )
- LastIndexOf( )
- Insert( )
- InsertRange( )
- Remove( )
- RemoveAt( )
- RemoveRange( )
- Reverse( )
- Sort( )
- ToArray( )

Clear(); Listedeki tüm elemanları silmek için kullanılır.

- Listedeki tüm elemanlar silinecektir.

```
// listedeki tüm elemanları siler
liste.Clear();
```

Contains()

- Listede bir elemanın olup olmadığına bakmak için kullanılır.
  - Sonuç true veya false olur.

```
// şehirler adında bir liste tanımlanıyor
List<string> Şehirler = new List<string>();

// şehirler listesine bazı şehirler ekleniyor
Şehirler.Add("Ankara");
Şehirler.Add("İstanbul");
...
// şehirler listesine daha fazla şehrin eklendiğini varsayalım…

// Bartın'ın eklenmiş olup olmadığına bak.
// Eğer eklenmiş ise "zaten eklenmiş" diye uyar, eklenmemiş ise ekle.
if (Şehirler.Contains("Bartın") == true)
    Console.WriteLine("Bartın zaten eklenmiş");
else
    Şehirler.Add("Bartın");
```

CopyTo()

- Listeyi veya listedeki bazı elemanları aynı tipten başka bir diziye kopyalar.
- Üç farklı kullanımı var:
  - CopyTo(dizi);
  - CopyTo(dizi, dizi_indexi);
  - CopyTo(index, dizi, dizi_indexi, adet)

```
// bir int listesi oluşturuluyor ve rakamlar ekleniyor.
List<int> liste = new List<int>();
liste.AddRange(new int[] { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 });

int[] dizi = new int[20];  // bir int dizisi tanımlanıyor.

// listenin elemanları diziye kopyalanıyor.
liste.CopyTo(dizi);

// listenin elemanları dizinin 3. indexinden sonraya kopyalanıyor.
liste.CopyTo(dizi, 3);

// listenin 2. indexinden sonraki 5 elamanı dizinin 4. indexinden sonraya kopyalanıyor.
liste.CopyTo(2, dizi, 4, 5);
```

IndexOf()

- Listeki herhangi bir elemanın ilk index değerini bulur.
  - Bu eleman birden fazla kez eklenmiş ise ilk index değeri bulunur.
  - Bu eleman listede hiç yoksa index değeri -1 olur.

```
// şehirler adında bir string listesi oluşturuluyor.
List<string> Şehirler = new List<string>();

// bu listeye bazı şehirler ekleniyor.
Şehirler.AddRange(new string[]{
    "Adana",  "Ankara", "Bartın", "İstanbul", "Zonguldak",
    "Rize", "Trabzon", "Kayseri", "Bartın", "Denizli"});

// şehirlerden "Bartın" ın ilk indexi aranıyor.
int index = Şehirler.IndexOf("Bartın"); // index 2 olur.

// şehirlerden "Karabük" ün indexi aranıyor.
index = Şehirler.IndexOf("Karabük"); // index -1 olur.
```

LastIndexOf()

- Listeki herhangi bir elemanın son index değerini bulur.
  - Bu eleman sadece bir kez eklenmiş ise ilk index değeri bulunur.
  - Bu eleman listede hiç yoksa index değeri -1 olur.

```
// şehirler adında bir string listesi oluşturuluyor.
List<string> Şehirler = new List<string>();

// bu listeye bazı şehirler ekleniyor.
Şehirler.AddRange(new string[]{
    "Adana",  "Ankara", "Bartın", "İstanbul", "Zonguldak",
    "Rize", "Trabzon", "Kayseri", "Bartın", "Denizli"});

// şehirlerden "Bartın" ın son indexi aranıyor.
int index = Şehirler.LastIndexOf("Bartın"); // index 8 olur.

// şehirlerden "Karabük" ün indexi aranıyor.
index = Şehirler.LastIndexOf("Karabük"); // index -1 olur.
```

Insert()

- Listenin belirtilen bir yerine eleman eklemek için kullanılır. (araya eleman ekleme)
  - Araya eleman eklenirken eklenecek index ile beraber belirtilir.
  - Belirtilen index listede yoksa hata verir!

```
// şehirler adında bir string listesi oluşturuluyor.
List<string> Şehirler = new List<string>();

// bu listeye bazı şehirler ekleniyor.
Şehirler.AddRange(new string[]{
    "Adana",  "Ankara", "Bartın", "İstanbul", "Zonguldak",
    "Rize", "Trabzon", "Kayseri", "Bartın", "Denizli"});

// 2. indexe "Ankara" ile "Bartın" arasına "Karabük" eklenecek.
Şehirler.Insert(2,"Karabük");

// Listenin en başına "Mersin" eklenecek.
Şehirler.Insert(0, "Mersin");
```

InsertRange()

- Listenin belirtilen bir yerine birden fazla eleman eklemek için kullanılır.
  - Elemanlar eklenirken eklenecek başlangıç indexi ile beraber belirtilir.
  - Belirtilen index listede yoksa hata verir!

```
// şehirler adında bir string listesi oluşturuluyor.
List<string> Şehirler = new List<string>();

// bu listeye bazı şehirler ekleniyor.
Şehirler.AddRange(new string[]{
    "Adana",  "Ankara", "Bartın", "İstanbul", "Zonguldak",
    "Rize", "Trabzon", "Kayseri", "Bartın", "Denizli"});

// belirtilen şehirler 3. indexe "Bartın" ile "İstanbul" arasına eklenecek.
Şehirler.InsertRange(3, new string[]{"Van", "Muş", "Kars"});

// belirtilen şehirler listenin en başına eklenecek.
Şehirler.InsertRange(0, new string[] { "Ordu", "Mardin", "Bursa" });
```

Remove()

- Listeden belirtilen bir elemanın silinmesi sağlanır.
  - Eleman listede birden fazla kez tekrarlanıyorsa sadece ilk geçtiği yerden silinir.
  - Tümü silinmek isteniyorsa döngü kullanılabilir.
  - Belirtilen eleman listede yoksa sorun olmaz.

```
// şehirler adında bir string listesi oluşturuluyor.
List<string> Şehirler = new List<string>();

// bu listeye bazı şehirler ekleniyor.
Şehirler.AddRange(new string[]{
    "Adana",  "Ankara", "Bartın", "İstanbul", "Zonguldak",
    "Rize", "Trabzon", "Kayseri", "Bartın", "Denizli"});

// sadece 2. index teki "Bartın" silinecek.
Şehirler.Remove("Bartın");

// şehirler listesinde "Bartın" olduğu sürece sil
while (Şehirler.Contains("Bartın") == true)
{
    Şehirler.Remove("Bartın");
}
```

RemoveAt()

- Listeden belirtilen index teki bir elemanın silinmesi sağlanır.
  - Belirtilen index listede yoksa hata verir!

```
// şehirler adında bir string listesi oluşturuluyor.
List<string> Şehirler = new List<string>();

// bu listeye bazı şehirler ekleniyor.
Şehirler.AddRange(new string[]{
    "Adana",  "Ankara", "Bartın", "İstanbul", "Zonguldak",
    "Rize", "Trabzon", "Kayseri", "Bartın", "Denizli"});

// sadece 2. index teki "Bartın" silinecektir.
Şehirler.RemoveAt(2);
```

RemoveRange()

- Listeden belirtilen index ten beşlayarak belirli sayıda elemanın silinmesi sağlanır.
  - Başlangıç indexi ve silinecek adet beraber belirtilir.
  - Belirtilen index listede yoksa hata verir!

```
// şehirler adında bir string listesi oluşturuluyor.
List<string> Şehirler = new List<string>();

// bu listeye bazı şehirler ekleniyor.
Şehirler.AddRange(new string[]{
    "Adana",  "Ankara", "Bartın", "İstanbul", "Zonguldak",
    "Rize", "Trabzon", "Kayseri", "Bartın", "Denizli"});

// listeden 2. index teki "Bartın" dan başlayıp
// 3 tane şehir ("Bartın", "İstanbul", "Zonguldak") silinecektir.
Şehirler.RemoveRange(2, 3);
```

Reverse()

- Listedeki elemanların sırasını tersine çevirir.

```
// şehirler adında bir string listesi oluşturuluyor.
List<string> Şehirler = new List<string>();

// bu listeye bazı şehirler ekleniyor.
Şehirler.AddRange(new string[]{
    "Adana",  "Ankara", "Bartın", "İstanbul", "Zonguldak",
    "Rize", "Trabzon", "Kayseri", "Bartın", "Denizli"});

// liste tersine çevir
Şehirler.Reverse();

// elemanları ekrana yazdır.
foreach(string s in Şehirler)
    Console.WriteLine(s);
```

```
Denizli
Bartın
Kayseri
Trabzon
Rize
Zonguldak
İstanbul
Bartın
Ankara
Adana
```

Sort()

- Listenin elemanlarını sıralar.
  - Sayısal elemanları küçükten büyüğe sıralar (0→9)
  - Alfabetik elemanları harf sırasına göre sıralar. (A→Z)

```
// şehirler adında bir string listesi oluşturuluyor.
List<string> Şehirler = new List<string>();

// bu listeye bazı şehirler ekleniyor.
Şehirler.AddRange(new string[]{
    "Adana",  "Ankara", "Bartın", "İstanbul", "Zonguldak",
    "Rize", "Trabzon", "Kayseri", "Bartın", "Denizli"});

// listeyi sırala
Şehirler.Sort();

// elemanları ekrana yazdır.
foreach(string s in Şehirler)
    Console.WriteLine(s);
```

```
Adana
Ankara
Bartın
Bartın
Denizli
İstanbul
Kayseri
Rize
Trabzon
Zonguldak
```

Z→A Sıralama

- Listenin elemanlarını büyükten küçüğe veya Z'den A'ya sıralamak istersek ne yapmalıyız.

```
// şehirler adında bir string listesi oluşturuluyor.
List<string> Şehirler = new List<string>();

// bu listeye bazı şehirler ekleniyor.
Şehirler.AddRange(new string[]{
    "Adana",  "Ankara", "Bartın", "İstanbul", "Zonguldak",
    "Rize", "Trabzon", "Kayseri", "Bartın", "Denizli"});

// listeyi A'dan Z'ye sırala
Şehirler.Sort();

// liste tersine çevir
Şehirler.Reverse();

// elemanları ekrana yazdır.
foreach(string s in Şehirler)
    Console.WriteLine(s);
```

```
Zonguldak
Trabzon
Rize
Kayseri
İstanbul
Denizli
Bartın
Bartın
Ankara
Adana
```

ToArray()

- Listeyi diziye çevirir.

```
// şehirler adında bir string listesi oluşturuluyor.
List<string> Şehirler = new List<string>();

// bu listeye bazı şehirler ekleniyor.
Şehirler.AddRange(new string[]{
    "Adana",  "Ankara", "Bartın", "İstanbul", "Zonguldak",
    "Rize", "Trabzon", "Kayseri", "Bartın", "Denizli"});

// listeyi diziye çevir.
string[] dizi = Şehirler.ToArray();

// dizinin eleman sayısı listenin eleman sayısı kadar
// olacak ve listenin elemanları diziye geçecektir.
```

Fonksiyonel Programlamanın Avantajları

- Kod tekrarlamalarını önler
- Kodumuzu küçük parçalara ayırıp daha kolay anlama ve modifiye edebiliriz.
- Doğruluk:
  - Bir kere fonksiyonumuzu yazıp test ettikten sonra her çağrışımızda doğru sonuç verecektir.
- Tekrar kullanma:
  - Artık fonksiyonumuzu istediğimiz kadar ve istediğimiz programlarda kullanabiliriz.

Fonksiyon Başlatma

- Bir Fonksiyon çağrılırken
  - Fonksiyon çağrısı yapılır
  - Kontrol fonksiyonun başında ki ilk ifadeye geçer
  - Fonksiyon adımları çalıştıktan sonra kontrol fonksiyonun çağrıldığı yerden sonraki ifadeye geri döner.

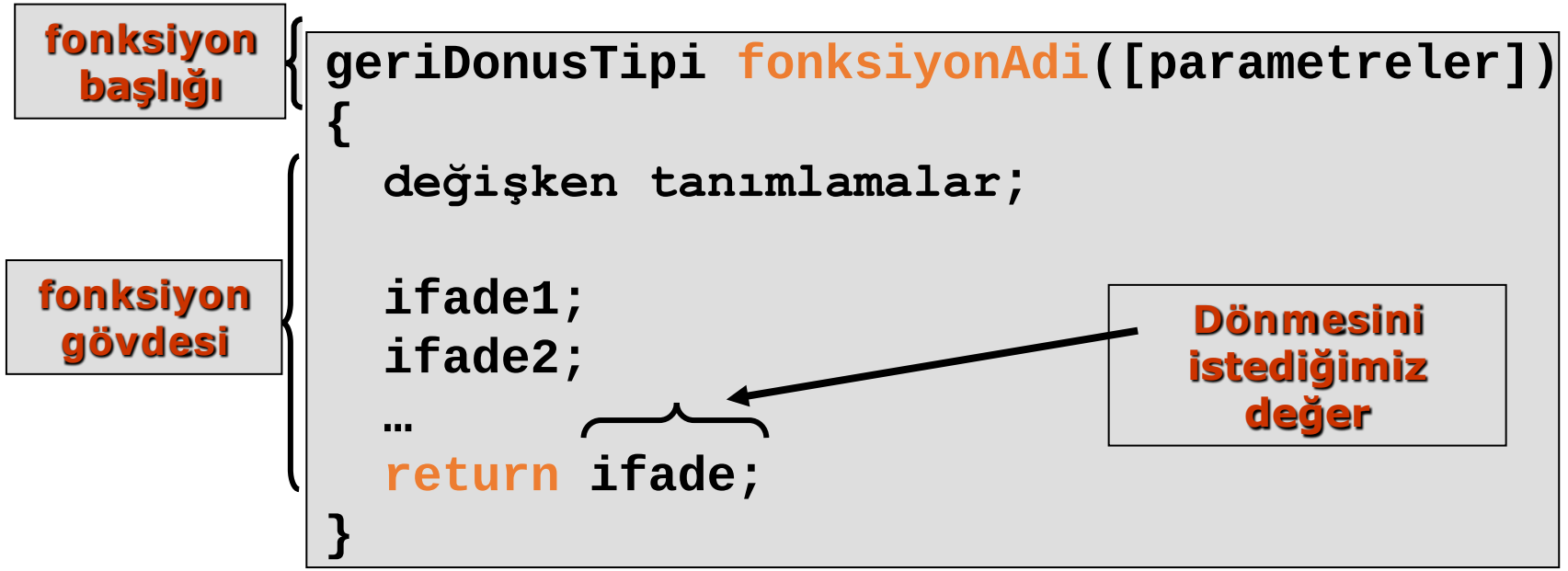
Fonksiyon Nedir?

- fonksiyon belli bir işi yapmak için kullanılan bir dizi ifadenin isimlendirilmesidir.
  - WriteLine ekrana mesaj yazdırır
  - ReadLine klavyeden bir girdi okur
  - ReadKey klavyeden basılan tuşu okur

- Bizim oluşturduğumuz fonksiyonlar:
  - ucgen '*' karakterini kullanarak ekrana üçgen çizer
  - kare '+' karakterini kullanarak ekrana kare çizer.
  - …

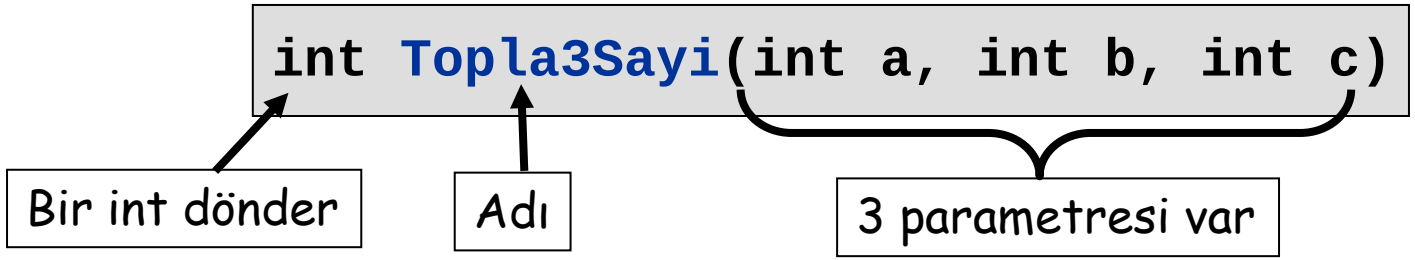
Her fonksiyonda bulunması gerekenler:

- fonksiyon başlığı
- Kod bloğu {….}
  - Fonksiyon gövdesini kapsar.



Fonksiyon Başlığı

- Geri dönüş tipi sonra fonksiyon adı ve opsiyonel olarak parantez içinde parametrelerden oluşur.
  - Parametreler fonksiyon çağrılırken program tarafından fonksiyona gönderilen verilerdir.



Örnek fonksiyon başlıkları

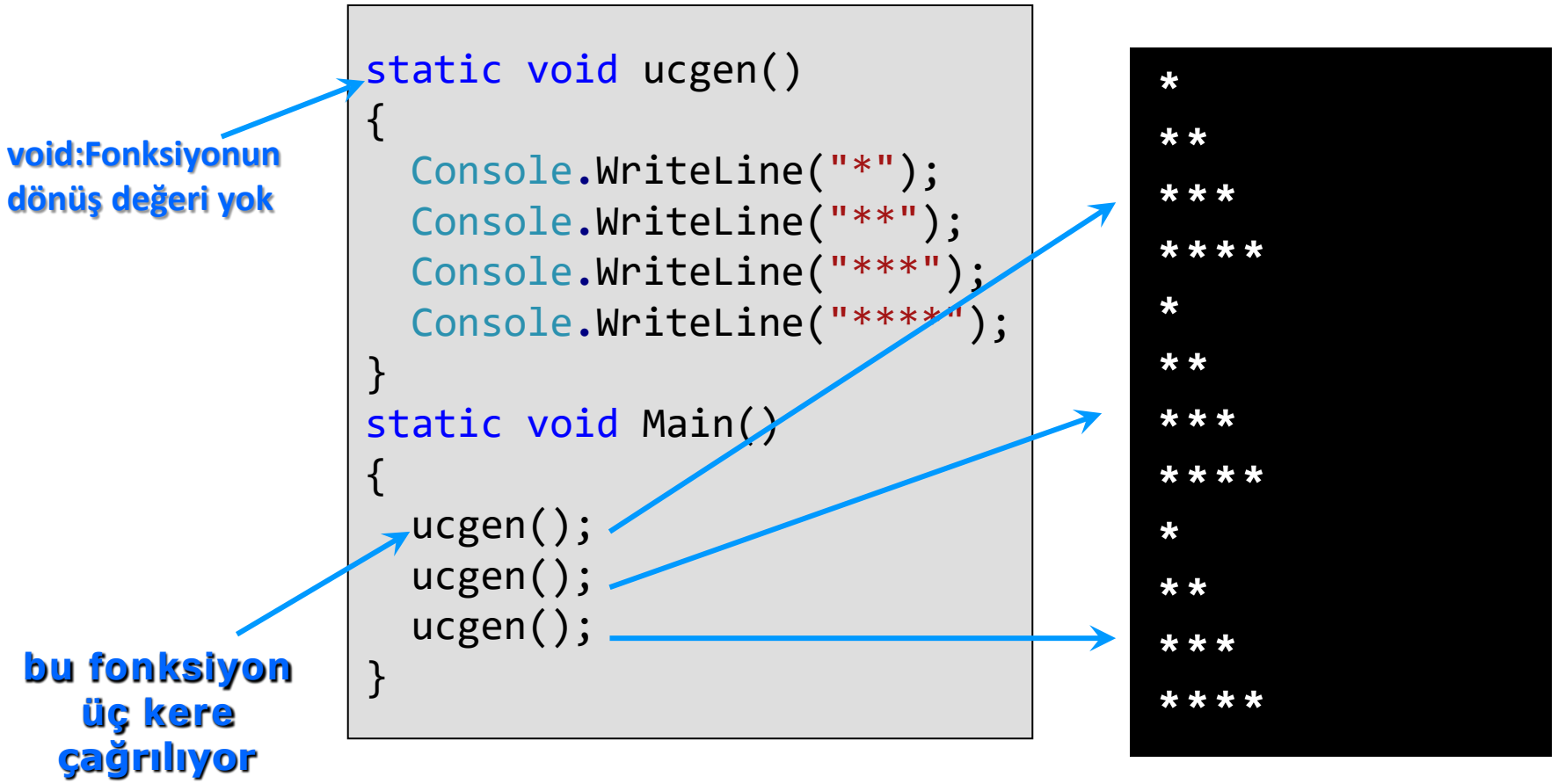
(1) Hiç parametresi olmayan ve değer döndürmeyen fonksiyon.
```
void Fonksiyon1()
```

(2) 1 int parametresi olan ve değer döndürmeyen fonksiyon.
```
void Fonksiyon2(int a)
```

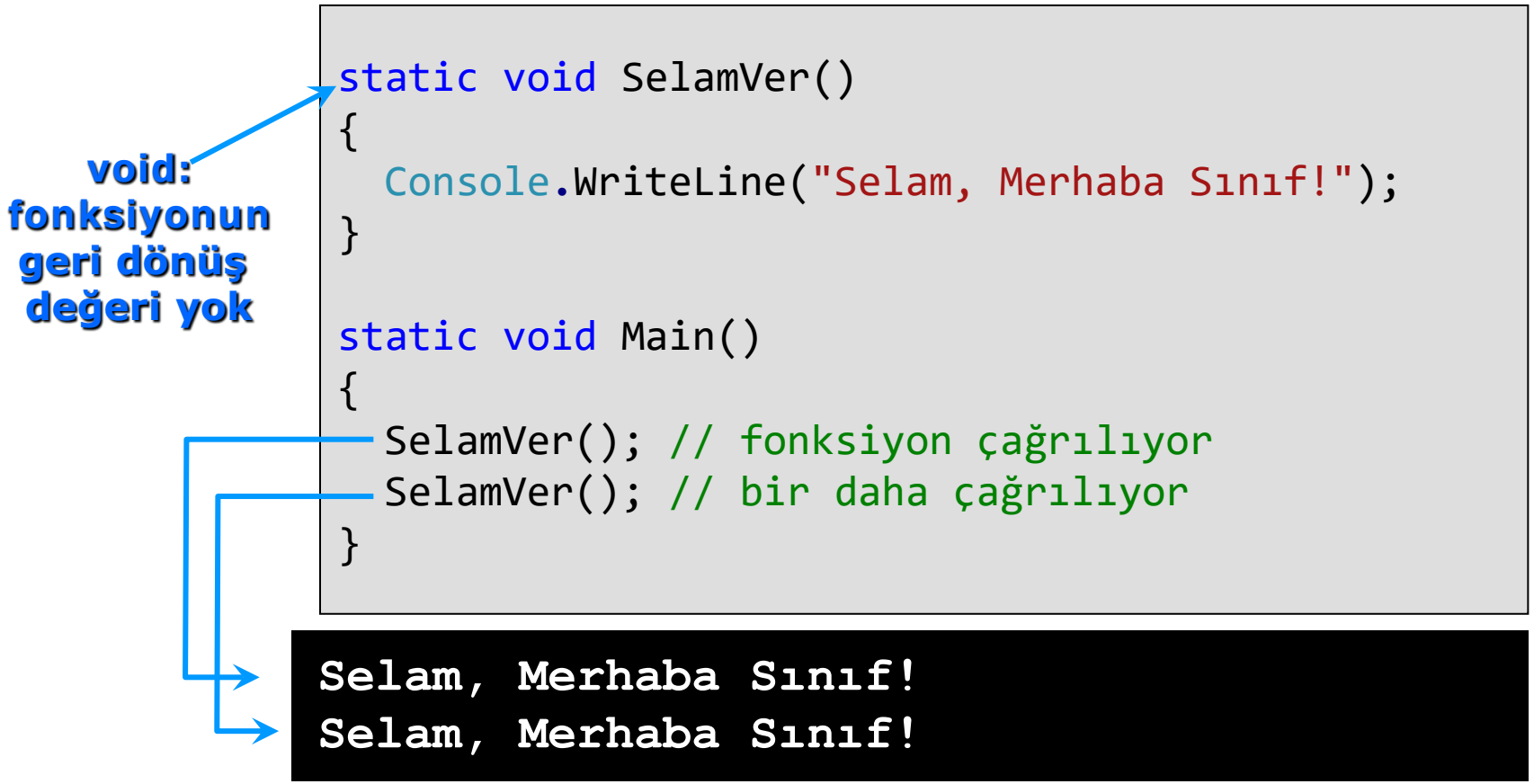
(3) Parametresi olmayan ve bir int döndüren fonksiyon.
```
int Fonksiyon3()
```

(4) iki parametresi olan ve bir int döndüren fonksiyon.
```
int Fonksiyon4(int a, int b)
```

Parametresi olmayan ve değer döndürmeyen fonksiyon(1)



Parametresi olmayan ve değer döndürmeyen fonksiyon(2)



Bir parametresi olan ve değer döndürmeyen fonksiyon(1)

```
static void nYazdir(int n)
{
  Console.WriteLine("n = {0}", n);
}

static void Main()
{
  nYazdir(573);
  nYazdir(-1234);
}
```

int n 573 olarak geçiliyor, ve fonksiyon çağırılıyor.

int n -1234 olarak geçiliyor, ve fonksiyon çağırılıyor.

```
n = 573
n = -1234
```

Bir parametresi olan ve değer döndürmeyen fonksiyon(2)

```
static void KuvvetAl(double x)
{
  double x2 = x * x;
  double x6 = x2 * x2 * x2;
  Console.WriteLine("x   = {0:F6}", x);
  Console.WriteLine("x^2 = {0:F6}", x2);
  Console.WriteLine("x^6 = {0:F6}", x6);
}

static void Main()
{
  KuvvetAl(1.5);
  Console.WriteLine("--------");
  KuvvetAl(0.11);
}
```

```
x   = 1,500000
x^2 = 2,250000
x^6 = 11,390625
--------
x   = 0,110000
x^2 = 0,012100
x^6 = 0,000002
```

double x 1.5 olarak geçiliyor, ve fonksiyon çağırılıyor.

double x 0.11olarak geçiliyor, ve fonksiyon çağırılıyor.

Bir parametresi olan ve değer döndürmeyen fonksiyon(3)

```
static void YildizYazdir(int n)
{
  for (int i = 0; i < n; i++)
    Console.Write("*");
  Console.Write("\n");
}

static void Main()
{
  YildizYazdir(1);
  YildizYazdir(2);
  YildizYazdir(3);
  YildizYazdir(4);
}
```

```
*
**
***
****
```

int n 1 olarak geçiliyor, ve fonksiyon çağırılıyor.

int n 2 olarak geçiliyor, ve fonksiyon çağırılıyor.

Aynı işi yapan Kod:

```
static void YildizYazdir(int n)
{
  for (int i = 0; i < n; i++)
    Console.Write("*");
  Console.Write("\n");
}

static void Main()
{
  for (int i = 1; i < 5;i++ )
    YildizYazdir(i);
}
```

```
*
**
***
****
```

int n, i değerini alarak geçiliyor, ve fonksiyon çağırılıyor.

Bir parametresi olan ve değer döndüren fonksiyon

```
// x ^ 5  döndürür
static double Kuvvet5(double x)
{
  double x5 = x * x * x * x * x;

  return x5;
}

static void Main()
{
  double p5 = Kuvvet5(1.5);
  Console.WriteLine("{0}", p5);
}
```

```
// 1 + 2+... + n  toplamı
static int Topla(int n)
{
  return n * (n + 1) / 2;
}

static void Main()
{
  int tp = Topla(5);
  Console.WriteLine("{0}", tp);
}
```

2 parametresi olan ve değer döndürmeyen fonksiyon

```
static void Yazdir(int n, char c)
{
  for (int i = 0; i < n; i++)
    Console.Write(c);

  Console.WriteLine();
}

static void Main()
{
  Yazdir(5, '*');
  Yazdir(10, '-');
  Yazdir(15, '=');
}
```

```
*****
----------
===============
```

Notlar

- Fonksiyonları kullanırken parametrelerini doğru sayıda, doğru sırada ve doğru türde vermeliyiz.
- Değer döndüren fonksiyonlarda return satırıyla belirtilen ifade, fonksiyonu oluştururken verilen türle uyumlu olmalıdır.
- Değer döndürmeyen (void ile belirtilmiş) fonksiyonlarda return komutu kullanılabilir, fakat herhangi bir bir ifadeyle kullanılamaz.
- Değer döndürmeyen fonksiyonlar bir değermiş gibi kullanılamaz.
- Fonksiyonlar değer döndürmeyebileceği gibi, parametre de almayabilirler.

Kombinasyon Hesabı

- Kombinasyon hesabı aşağıdaki formül ile yapılır:

$$c(n,r) = \frac{n!}{(n-r)!\,r!}$$

- Dikkat edilirse üç defa faktöriyel hesabı yapılmaktadır. **(** n!, (n-r)! ve r! **)**
- Eğer fonksiyon kullanılmazsa üç defa faktöriyel hesabı yapmak için benzer kodlar yazılacaktır.
- Eğer faktöriyel hesabı yapacak bir fonksiyon yazılırsa aynı fonksiyon üç defa kullanılabilir.

Fonksiyon kullanmadan kombinasyon hesabı

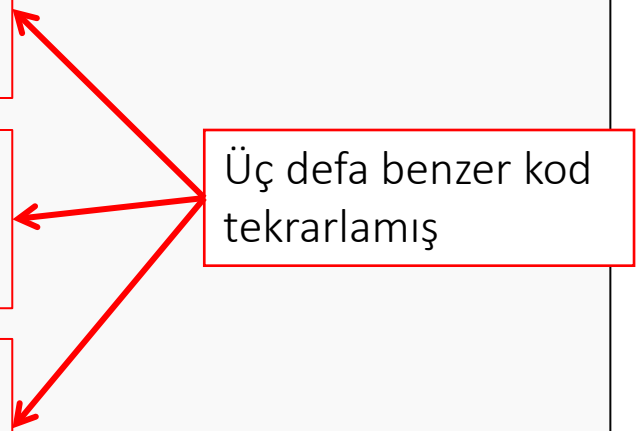
```
static void Main()
{
    int n = 5, r = 3;

    int n_fakt = 1;
    for (int i = 1; i <= n; i++)
        n_fakt *= i;

    int r_fakt = 1;
    for (int i = 1; i <= r; i++)
        r_fakt *= i;

    int nr_fakt = 1;
    for (int i = 1; i <= (n - r); i++)
        nr_fakt *= i;

    int comb = n_fakt / (nr_fakt * r_fakt);
    Console.WriteLine("C({0},{1}) = {2}", n, r, comb);
}
```



Fonksiyon kullanarak kombinasyon hesabı

```
static void Main()
{
    int n = 5, r = 3;

    int comb = fakt(n) / (fakt(n - r) * fakt(r));
    Console.WriteLine("C({0},{1}) = {2}", n, r, comb);
 }

static int fakt(int s)
{
    int fk = 1;
    for (int i = 1; i <= s; i++)
        fk *= i;

    return fk;
}
```

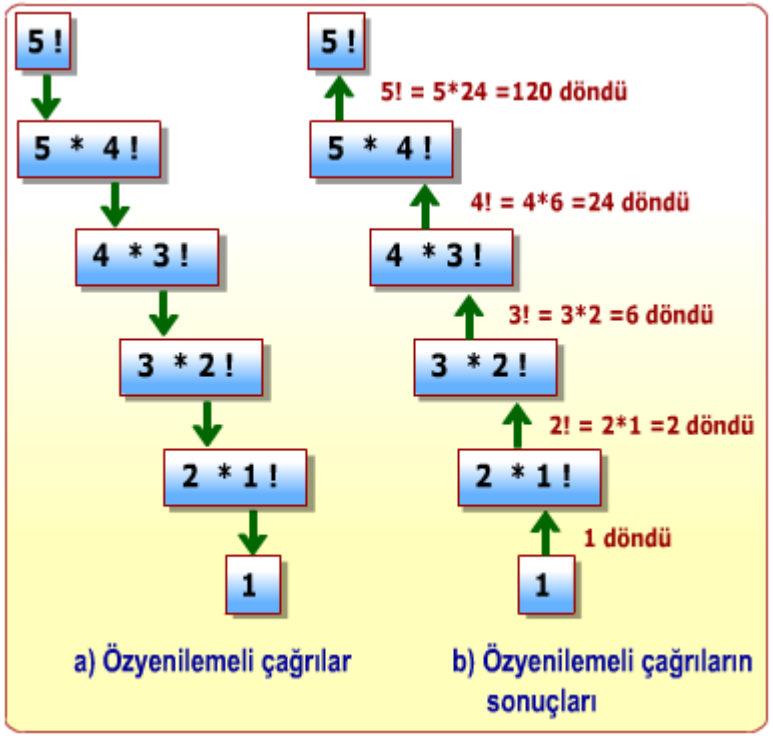
Özyenilemeli(Recursive) Yordamlar

- Özyenilemeli (recursive) yordamlar tekrar tekrar kendilerini çağırırlar.
- Özyenilemeli algoritmalarda problemin en basit hali için çözüm bulunur. Bu en basit duruma temel durum (base case) denir. Eğer yordam temel durum için çağırılırsa sonuç döndürülür. Eğer temel durum için değil, diğer durumlar için çağrılırsa temel durumdan yararlanılarak, problemi çözmeye çalışır yani kendini çağırır. Bu tür durumlar için yapılan her çağrı recursion step olarak adlandırılır.
- Basit bir örnek: Faktoriyel
  - n! = n * (n-1) * (n-2) * .... * 3 . 2 . 1

Recursive Yordam Örnek I

- Klasik Süreç (For döngüsü ile çözüm)

```
static void Main(string[] args)
{
    int n = 5;
    int faktoriyel = 1;
    for (int i = n; i >= 1; i--)
        faktoriyel *= i;
}
```



- Recursive Çözüm

```
static int Faktoriyel(int sayi)
{
    if (sayi <= 1) // Eğer temel durumsa 1 dönder
        return 1;
    else                        // Temel durum değilse n * (n -1)! bul.
        return sayi * Faktoriyel(sayi - 1);
}
```

Recursive Yordam Örnek II

- Fibonacci Serisi: Bir dizinin her elemanı bir önceki iki elemanın toplamını verir.
  - 0 1 1 2 3 5 8 13 …

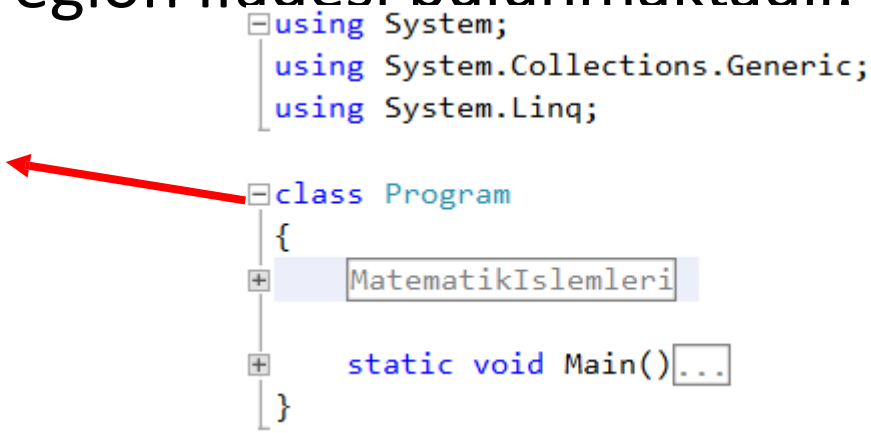
```
//Döngülü
static int Fibonacci(int n)
{
    int a = 0;
    int b = 1;
    for (int i = 0; i < n; i++)
    {
        int temp = a;
        a = b;
        b = temp + b;
    }
    return a;
}
```

```
//Recursive
static int fib(int n)
{
    if (n == 0 || n == 1)
        return n;
    else
        return (fib(n - 1) + fib(n - 2));
}
```

Region Özelliği

- Region, kodları daha düzenli bir yapıda saklamak ve kodların okunurluğunu arttırmak için kullanılmaktadır.
- [illegible]nek istenilen kodun üst tarafında #region ,
- [illegible]e #endregion ifadesi bulunmaktadır.

```
#region MatematikIslemleri
//Dört işlem yapan bölüm işlemleri
static int topla(int x, int y)
{
    return x + y;
}
static int cikar(int x, int y)
{
    return x - y;
}
static int carpma(int x, int y)
{
    return x * y;
}
static double bolme(int x, int y)
{
    return (double)x / y;
}
#endregion
```



Temel I/O İşlemleri – Dizin, Dosya

- Bilgisayar ile yaptığımız her işlem aslında bir girdi/çıktı işlemidir. Klavyeden veri girişi, fare ile program açmak, ekrana veya yazıcıya belge yazdırmak, dosya kaydetmek I/O işlemlerine birer örnektir.
- Bu bölümde, I/O işlemleri ise .Net System.IO isim alanı altındaki sınıflarla yapabileceğimiz dosya ve dizin okuma/yazma işlemleridir.

Temel I/O İşlemleri - Dizin (Directory)

- Directory sınıfı klasör oluşturma, silme, yeniden adlandırma ve taşıma gibi klasör işlemleri yapabilmemizi sağlayan statik metotları barındırır.
- Eğer bir klasör üzerinde sadece bir işlem yapmak istiyorsanız, instance alınmadan kullanıldığı ve aşırı yüklenmeden kaçındığı için bu sınıfı kullanmanız daha faydalı olacaktır.

Temel I/O İşlemleri - Dizin (Directory)

```
// Gerekli dosya konumlarını tanımlıyoruz.
string path = @"C:\Test\TestDizini";
string target = @"C:\Test\HedefDizini";

// Exist Static Metodu ile dizinin var olup olmadığının kontrolü.
if (!Directory.Exists(path))
{
    // CreateDirectory metodu ile dizin oluşturma işlemi.
    Directory.CreateDirectory(path);
    Console.WriteLine("Oluşturulma Tarihi: " + Directory.GetCreationTime(path)); //oluşturma tarigi
    Console.WriteLine("Son Erişim Tarihi: " + Directory.GetLastAccessTime(path)); //son erişim tarihi
    Console.WriteLine("Son Değiştirilme Tarihi: " + Directory.GetLastWriteTime(path)); //son yazma tarihi
    Console.WriteLine("Bulunduğu Dizinin Adı: " + Directory.GetParent(path)); //bulunduğu dizi
    Console.ReadLine();
}
if (Directory.Exists(target))
{
    // Delete metodu ile dizin silme işlemi
    Directory.Delete(target, true);
}
//Move metodu ile dizini taşıma işlemi.
Directory.Move(path, target);
//GetDirectories ile dizindeki klasörlerin seçimi.
string[] directories = Directory.GetDirectories(@"C:\Test\");
foreach (string dir in directories)
{
    Console.WriteLine(dir);
}
// Yeni bir Metin Belgesi oluşturma.
File.CreateText(target + @"\NewFile.txt");
// GetFiles ile dizindeki dosyaların seçimi.
Console.WriteLine("{0} dizinindeki dosya sayısı: {1}", target, Directory.GetFiles(target).Length);
Console.ReadLine();
```

DirectoryInfo

- DirectoryInfo, Directory sınıfına benzer metotları olan ve yine klasör işlemlerinde kullandığımız instance alınabilir bir sınıf olup bir çok özelliği vardır. Bir objeyi birden fazla kullanacağımız durumlarda, DirectoryInfo ile instance alıp çalışmamız performans açısından daha etkilidir. Uygulama içerisinde bu özellikleri kullanarak inceleyelim. (NYP uygun bir yapı.)

DirectoryInfo

```
// Kullanılacak dizini tanımlama.
DirectoryInfo NewDir = new DirectoryInfo(@"C:\Test\TestDizini");
// Dizinin var olup olmadığının kontrolü.
if (NewDir.Exists)
{
    Console.WriteLine("Dizin Mevcut.");
    // Dizin silme işlemi.
    NewDir.Delete();
    Console.WriteLine("Dizin Silindi..");
}
 // Yeni dizin oluşturma.
NewDir.Create();
Console.WriteLine("Klasör oluşturuldu.");
Console.WriteLine("Oluşturulma Tarihi: " + NewDir.CreationTime);
Console.WriteLine("Bulundugu Dizinin Adı: " + NewDir.Parent);
Console.WriteLine("Dizinin Adı: " + NewDir.Name);
Console.WriteLine("Dizinin Tam Adı : " + NewDir.FullName);
Console.WriteLine("Son Erişim Tarihi : " + NewDir.LastAccessTime);
Console.WriteLine("Son Değiştirilme Tarihi : " + NewDir.LastWriteTime);
Console.ReadLine();
//Dizin içerisinde alt dizin oluşturma.
DirectoryInfo SubDir = NewDir.CreateSubdirectory("AltDizin");
//Alt Dizin içerisinde alt dizin oluşturma.
SubDir.CreateSubdirectory("AltDizin2");
 // GetFiles ile dizindeki dosyaların seçimi.
Console.WriteLine("{0} dizinindeki dosya sayısı: {1}",
NewDir.FullName, NewDir.GetFiles().Length);
// GetDirectories ile dizindeki klasörlerin seçimi.
Console.WriteLine("{0} dizinindeki klasör sayısı: {1}", NewDir.FullName, NewDir.GetDirectories().Length);
Console.ReadLine();
```

Dosya (File)

- File sınıfı ve Directory sınıfının bazı metotları aynıdır ve tek farkı klasörler yerine dosyalar üzerinde işlem yapmasıdır.
- Bu metotlar şunlardır:
  - Exists(),
  - Delete(),
  - GetCreationTime(),
  - GetLastAccessTime(),
  - GetLastWriteTime(),
  - Move(),
  - SetCreationTime(),
  - SetLastAccessTime(),
  - SetLastWriteTime()
- Ortak metotların yanı sıra dosyalar üzerinde farklı işlemler yapabilmesi için farklı metotları mevcuttur. Bu metotlar da FileInfo sınıfı instance metotları ile aynıdır.

FileInfo

- File sınıfı gibi dosyalar üzerinde işlem yapabilen ve de benzer metotları bulunan bir sınıftır.
- File sınıfı ile işlem yaparken her daim dosya yolunu girmek gerekirken, FileInfo ile instance alındıktan sonra tekrar kullanımı daha efektiftir.

```
FileInfo NewFile = new FileInfo(@"C:\Test\HedefDizini\NewFile.txt");
if (NewFile.Exists) // File sınıfı:  if(File.Exists(@"C:\Test\NewFile.txt"))
{
    // Dosya silme işlemi.
    NewFile.Delete();   // File sınıfı: File.Delete(@"C:\Test\NewFile.txt"))
}
// Dosya oluşturma.
NewFile.CreateText(); // File sınıfı: File.CreateText(@"C:\Test\NewFile.txt"))
Console.WriteLine("Oluşturulma Tarihi: " + NewFile.CreationTime);
Console.WriteLine("Son Erişim Tarihi : " + NewFile.LastAccessTime);
Console.WriteLine("Son Değiştirilme Tarihi : " + NewFile.LastWriteTime);
Console.WriteLine("Bulundugu Dizinin Adı: " + NewFile.DirectoryName);
Console.WriteLine("Dosyanın Adı: " + NewFile.Name);
Console.WriteLine("Dosyanın Tam Adı: " + NewFile.FullName);
Console.WriteLine("Dosyanın Uzantısı: " + NewFile.Extension);
// Dosya Kopyalama
NewFile.CopyTo(@"C:\Test\");
Console.ReadLine();
```

FileStream

- Okuma ve yazma işlemlerinde dosya açarken kullandığımız bir sınıftır.
- Bu sınıf ile Byte düzeyinde okuma ve yazma işlemleri de yapılabilir fakat bu işlemleri etkin olarak StreamReader ve StreamWriter sınıfları ile metin düzeyinde yapmamız daha efektif olacaktır.
- Bir FileStream nesnesi oluşturulurken FileMode, FileAccess ve FileShare enumları ile değişik yollarda yaratılabilir.

```
string adres = @"C:\TestDizini\NewFile.txt";
FileStream fs = new FileStream(adres, FileMode.OpenOrCreate);
FileStream fs2 = new FileStream(adres, FileMode.Append, FileAccess.Write);
FileStream fs3 = new FileStream(adres, FileMode.Truncate, FileAccess.Write, FileShare.None);
// FileStream akımının tamponu temizlenir
fs.Flush();
// FileStream tarafından kullanılan kaynaklar boşaltılır.
fs.Close();
```

FileStream

- FileMode enum
  - Append: Açılan dosyanın sonuna ekleme yapmak için kullanılır. Eğer dosya bulunmazsa oluşturulur.
  - Create: Yeni dosya oluşturmak için kullanılır. Zaten dosya varsa üzerine yazılır.
  - CreateNew: Yeni dosya oluşturmak için kullanılır, belirtilen dosya mevcutsa çalışma zamanı hatası verdirir.
  - Open: Dosyayı açmak için kullanılır.
  - OpenOrCreate: Belirtilen dosya varsa açılır, yoksa yenisi oluşturulur.
  - Truncate: Belirtilen dosya açılır ve içi tamamen silinir.
- FileAccess enum
  - Read: Dosya okumak için kullanılır.
  - ReadWrite: Dosya okunmak ve yazılmak üzere açılır.
  - Write: Dosya sadece yazılmak için açılır.
- FileShare enum
  - Inheritable: Dosyanın child (yavru) işlemler tarafından türetilebilmesini sağlar.
  - None: Dosyanın aynı anda başka işlemler tarafından açılmasını engeller.
  - Read: Dosyanın aynı anda başka işlemlerce de açılabilmesini sağlar.
  - ReadWrite: Dosyanın aynı anda başka işlemlerce de açılıp, okunup, yazılabilmesini sağlar.
  - Write: Dosyaya aynı anda başka işlemlerce yazılabilmesini sağlar.

StreamReader

- StreamReader, metin dosyalarını okur. System.IO namespace içinde bulunur.
- Using statement ile kullanılır. Bu yapı sistem kaynaklarını elden çıkarmada yardımcı olur.

```
//using olmazsa dispose ve close olmalı
StreamReader reader01 = new StreamReader("file.txt");
string line = reader01.ReadLine();
reader01.Close();
reader01.Dispose();
Console.WriteLine(line);

//using olunca dispose ve close'a gerek yok
using (StreamReader reader02 = new StreamReader("file.txt"))
{
    line = reader02.ReadLine();
}
Console.WriteLine(line);
```

StreamReader

```
//ReadToEnd son satıra kadar okur
string line;
using (StreamReader reader = new StreamReader("file.txt"))
{
    line = reader.ReadToEnd();
}
Console.WriteLine(line);

//ReadToEnd olmasaydı, döngü açıp okuma null oluncaya kadar devam edecektik.
using (StreamReader reader = new StreamReader("file.txt"))
{
    while (true)
    {
        line = reader.ReadLine();
        if (line == null)
        {
            break;
        }
        Console.WriteLine(line); // Use line.
    }
}
```

StreamReader

- Sonuçları satır satır List nesnesine aktarma.

```
List<string> list = new List<string>();
using (StreamReader reader = new StreamReader("file.txt"))
{
    string line;
    while ((line = reader.ReadLine()) != null)
    {
        list.Add(line); //List'e ekle
        Console.WriteLine(line); //Console'a yaz
    }
}
```

StreamWriter

- Dosyaya metin yazma işleminde kullanılır.

```
//Mevcut dosya varsa üstüne açar
using (StreamWriter writer =  new StreamWriter("important.txt"))
{
    writer.Write("Word ");
    writer.WriteLine("word 2");
    writer.WriteLine("Line");
}
//Dosya yoksa yeni açar
using (StreamWriter writer =  new StreamWriter("C:\\log.txt", true))
{
    writer.WriteLine("Important data line 1");
}
//Mevcut dosyaya satır ekler
using (StreamWriter writer =  new StreamWriter("C:\\log.txt", true))
{
    writer.WriteLine("Line 2");
}
```

String Metotları

- Compare
- CompareOrdinal
- CompareTo
- Concat
- Contains
- Copy
- CopyTo
- EndsWith
- Empty
- Equals
- Format
- GetEnumerator
- GetHashCode
- IndexOf
- IndexOfAny
- Insert
- Intern
- IsInterned
- IsNormalized
- IsNullOrEmpty
- IsNullOrWhiteSpace
- Join
- LastIndexOf
- LastIndexOfAny
- Length
- Normalize
- PadLeft
- PadRight
- Remove
- Replace
- Split
- StartsWith
- Substring
- ToCharArray
- ToLower
- ToLowerInvariant
- ToString
- ToUpper
- ToUpperInvariant
- Trim
- TrimEnd
- TrimStart

String Metotları

```
string input = "OneTwoThree";
//İlk üç karakter. Karakter başlangıcı ve kaç karakter okunacağı
string sub = input.Substring(0, 3);
Console.WriteLine("Substring: {0}", sub);

input = "_::_pagitica";
Console.WriteLine(input);
//Karakter veya karakter grubu değişirmede kullanılır
string output = input.Replace("_::_", "Areo");
Console.WriteLine(output);

string s = "there is a cat";
//Split işlemi belli bir string'e göre string'i bölüp diziye aktarır
//Aşağıdaki örnekte boşluk karakterine bölme işlemi yapılmıştır
string[] words = s.Split(' ');
foreach (string word in words)
{
    Console.WriteLine(word);
}
```

Diğer metotların hepsini internette bulabilirsiniz. String üzerine bir işleme gerek kaldığınızda dünyayı yeniden keşfetmeyin, hazır metotların size ne getireceğine bir bakın.

Oyun

- Sayı tahmin oyunu yazın. 1 ile 100 arasında bir sayı girin. Arkadaşınız o sayıyı bilgisayarın önerilerine göre tahmin etmeye çalışsın.
  - Sayıya 5 yaklaşıldıysa: Çok sıcak
  - Sayıya 10 yaklaşıldıysa: Sıcak
  - Sayıya 15 yaklaşıldıysa: Ehh işte
  - Sayıya 20 varsa: Soğuk
  - Sayıya 20'den fazla değerler için: Çok soğuk
  - Sayıya 50'den fazla değerler için: Çooookkk soğuk
- Komutları ile kullanıcıyı yönlendirecek. Ve kullanıcı sayıyı bulduğunda kaç tahminde bulunduğu ekrana yazacak programı yazınız.